

## Homework 2

### 1 Invariance and Equivariance of GNNs

**Permutation Matrix** A permutation  $\pi$  of  $m$  element is defined by  $\begin{pmatrix} 1 & 2 & \cdots & m \\ \pi(1) & \pi(2) & \cdots & \pi(m) \end{pmatrix}$ , where the  $i$ -th element is permuted to  $\pi(i)$ . The permutation matrix  $P_\pi = (p_{ij})$  is defined by  $p_{ij} = 1$  if  $j = \pi(i)$  and  $p_{ij} = 0$  otherwise. For example, the permutation matrix  $P_\pi$  corresponding

to the permutation  $\pi = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 4 & 3 & 1 \end{pmatrix}$  is  $\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$ .

An important property of permutation matrices is that they are orthogonal matrices (i.e.,  $P_\pi P_\pi^\top = I$ ). Therefore, we have  $P_\pi^{-1} = P_\pi^\top$ .

**Invariance and Equivariance** Consider  $X \in \mathbb{R}^{n \times d}$ , a function  $f(X)$  is **permutation invariant** if, for all permutation matrices  $P \in \mathbb{R}^{n \times n}$ :  $f(PX) = f(X)$  holds. Accordingly, we say that  $f(X)$  is **permutation equivariant** if, for all permutation matrices  $P \in \mathbb{R}^{n \times n}$ :  $f(PX) = Pf(X)$  holds. Note that in the case of a graph neural network, function  $f$  typically takes in adjacency matrix  $A$  and node features  $X$  parameterized by a learned weight matrix  $W$ .

1. If there is a permutation  $P$  applied to the nodes, the original features  $X \in \mathbb{R}^{n \times d}$  is changed to  $PX \in \mathbb{R}^{n \times d}$ , where  $n$  and  $d$  are the number of nodes and feature dimensions. Node permutations also accordingly act on the edges. **Question:** Write out how the adjacency matrix  $A \in \mathbb{R}^{n \times n}$  changes accordingly after node permutation.
2. **Question:** Consider the message passing function of a graph convolutional layer:  $H^{(l+1)} = \sigma(AH^{(l)}W_l^\top)$ ,  $W_l^\top$  is the weight matrix. Prove that a graph convolutional layer is permutation equivariant.
3. **Question:** Prove that if each layer of the network is permutation equivariant, then the network as a whole is permutation equivariant. (**Hint:** consider each layer as a function, then, stacking layers is equivalent to function composition. Prove the resulting composite function is permutation equivariant)

### 2 Laplacian Matrix over Graph

The graph Laplacian matrix is the ‘discrete’ version of the Laplacian operator over graphs. You might remember from multi-variable calculus, that in Euclidean space, the Laplace operator is the

divergence of the gradient of a function, i.e.  $\Delta f = \text{div}(\nabla(f))$ , where  $f$  is a function that maps  $n$ -dimensional real vector to a real number:  $f : \mathbb{R}^d \mapsto \mathbb{R}$ . Here, we want to extend this definition of Laplace operator to functions defined on graphs (instead of Euclidean space).

**Step1: functions over the graph:** A function on graph  $f$  is defined as a mapping from every

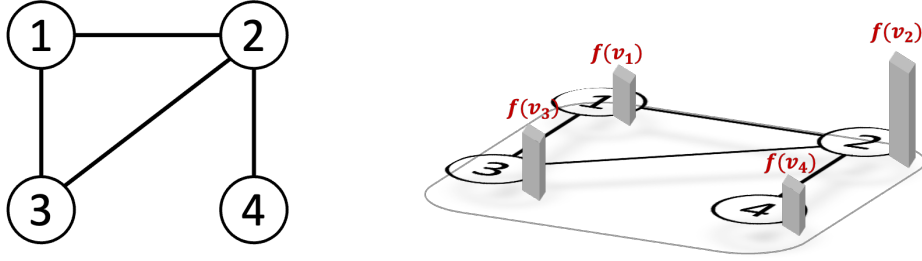


Figure 1: Graph and its associated function

node in the graph to a real vector space:  $f : V \mapsto \mathbb{R}^d$ . It can be 1-dimensional or a vector. In our demonstration, we use columns to represent the 1-dimensional function values over the nodes. That means, we define  $f$  as  $f : V \mapsto \mathbb{R}$ .

**Step2: define the gradient:** The gradient operator gives the derivative of the function along each direction. In a graph, gradients are defined over each edge. Thus, gradient of the function along the edge  $e = (v_i, v_j)$  is given by  $\nabla f|_e = f(v_i) - f(v_j)$ .

**Step3: define the divergence of graph gradient:** In a graph, the gradients over edges associated with a node construct a vector field over this node. Divergence at this node gives net outward flux of a vector field, that is,  $\text{div}(\nabla f)(v_i) = \sum_{(v_i, v_j) \in \mathcal{E}} (f(v_i) - f(v_j))$ .

1. Graph Laplacian operator  $L$  is defined by the divergence of graph gradient. When applying  $L$  to a function  $f$  defined on all vertices of a graph, we have

$$Lf(v_i) = \text{div}(\nabla f)(v_i)$$

Actually, applying the Laplacian operator  $L$  to the function  $f$  is equivalent to a matrix product with the vector  $\mathbf{f}$  (a vector with  $f(v_i)$  as the  $i$ -th entry). **Question:** Use the degree matrix  $D$  and adjacency matrix  $A$  to represent the Laplacian matrix.

2. **Question:** Write out the Laplacian matrix of the graph in Figure 1.
3. Normalized Laplacian matrix  $\tilde{L} = D^{-1/2}LD^{-1/2}$ , and normalized adjacency matrix  $\tilde{A} = D^{-1/2}AD^{-1/2}$ . **Question:** Figure out the relation between  $\tilde{L}$  and  $\tilde{A}$ . Rewrite the message passing function of a graph convolutional layer  $H^{(l+1)} = \sigma(\tilde{A}H^{(l)}W_l^\top)$  with normalized Laplacian matrix  $\tilde{L}$ .
4. Over-smoothing phenomenon means the node embeddings  $f(v_i)$  tend to become similar and indistinguishable, which is common after many layers of message passing in a GNN. Dirichlet energy is widely used to measure the degree of over-smoothing, which is defined as  $E(\mathbf{f}) = \text{trace}(\mathbf{f}^\top L \mathbf{f})$ . **Question:** Explain why the over-smoothing phenomenon corresponds to a small value of Dirichlet energy (high-level intuition is sufficient).

### 3 Graph Partition

Graph partitioning is the process of dividing a graph into smaller, disjoint subgraphs or "partitions" while minimizing the number of edges between them. The primary objective is to ensure that the connections (or edges) between different partitions are minimized. This technique has been widely applied in many scalable Graph Neural Network (GNN) algorithms, such as ClusterGCN.

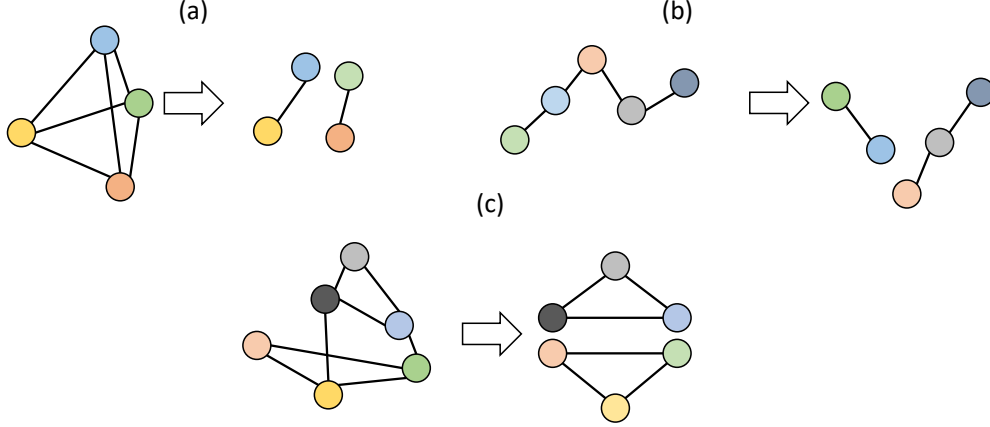


Figure 2: Three examples of graph partitioning.

To evaluate the quality of a partition, researchers have developed several metrics based on the characteristics of the partitions and the edges connecting them. An undirected graph with  $N$  nodes is defined as  $G = (V, E)$ , where  $V = \{v_i \mid 1 \leq i \leq N\}$  denotes a set of nodes and  $E = \{e_{ij} \mid v_i, v_j \in V\}$  is the set of edges connecting nodes. Given two partitioned graphs with separate node sets  $V_1$  and  $V_2$ , we define the following four metrics to evaluate their quality:

- **Cut:** Measures the total number of edges crossing between partitions, i.e.,

$$f_{\text{cut}}(V_1, V_2) = \sum_{v_i \in V_1, v_j \in V_2} e_{ij}$$

- **Ratio cut:** Normalizes the cut size by partition sizes, i.e.,

$$f_{\text{rcut}}(V_1, V_2) = \frac{f_{\text{cut}}(V_1, V_2)}{|V_1|} + \frac{f_{\text{cut}}(V_1, V_2)}{|V_2|}$$

where  $|V_1|$  and  $|V_2|$  denotes the number of nodes in  $V_1$  and  $V_2$ .

- **Normalized cut:** Normalizes the cut size by the total connectivity of the partitions, i.e.,

$$f_{\text{ncut}}(V_1, V_2) = \frac{f_{\text{cut}}(V_1, V_2)}{\text{vol}(V_1)} + \frac{f_{\text{cut}}(V_1, V_2)}{\text{vol}(V_2)}$$

where  $\text{vol}(V_1)$  and  $\text{vol}(V_2)$  denotes the sum of node degrees in  $V_1$  and  $V_2$ . Note that this considers the degree of each node in  $V_1$  and  $V_2$  within the original graph  $V$ .

- **Conductance:** Evaluates how well a subset is connected internally relative to the whole graph, i.e.,

$$f_{\text{cond}}(V_1, V_2) = \frac{f_{\text{cut}}(V_1, V_2)}{\min(\text{vol}(V_1), \text{vol}(V_2))}$$

- Figure 2 presents three examples of graph partitioning, where the original graph is divided into two subgraphs. **Question:** Evaluate the resulting partitions in each example by computing the relevant partition metrics.
- Question:** Explain why partitioning a graph by minimizing the cut alone can easily lead to highly unbalanced partitions.
- Question:** Prove that for a regular graph (where all nodes have the same degree), partitioning the graph into two subgraphs by minimizing the ratio cut is equivalent to minimizing the normalized cut.
- Given an undirected graph  $G = (V, E)$ , suppose we partition the node set  $V$  into two disjoint subsets  $V_1$  and  $V_2$ . Let  $d_V(u)$  as the number of connections between  $u$  and the nodes in  $V$ , i.e.,  $d_V(u) = |\{v \in V | (u, v) \in E\}|$ . Note that  $u$  does not necessarily need to be a node of  $V$ .  
**Question:** Prove that  $f_{\text{cond}}(V_1, V_2) = 1 - \frac{\sum_{u \in V_1} d_{V_1}(u)}{\sum_{u \in V_1} d_V(u)}$  when  $\text{vol}(V_1) < \text{vol}(V_2)$ :

## 4 Recommendation System

GNNs have demonstrated many successful applications in recommendation systems. In this context, user-item interactions can be modeled as a bipartite graph, where nodes represent users and items, and edges capture interactions between them. Consider a bipartite user-item graph  $G = (V, E)$  where  $V$  is the node set and  $E$  is the edge set, let  $\mathbf{A} \in \mathbb{R}^{|V| \times |V|}$  be its unnormalized adjacency matrix without self-loop connection, and  $\mathbf{D} \in \mathbb{R}^{|V| \times |V|}$  be its degree matrix.

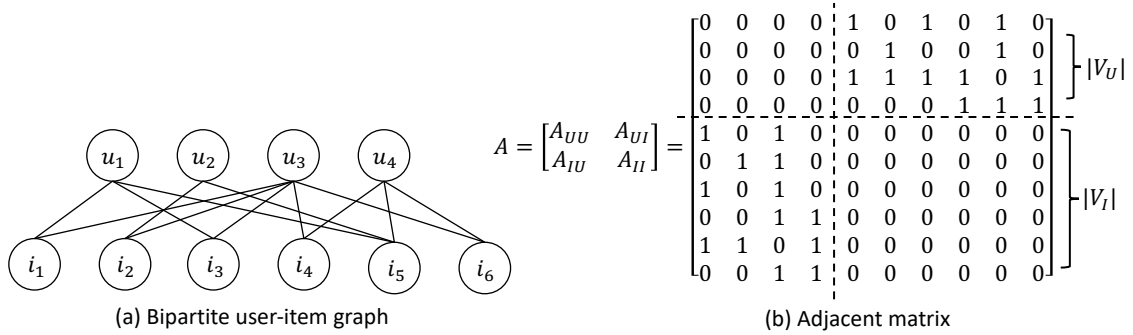


Figure 3: An example of a bipartite graph.

- A user-item bipartite graph comprises edges that connect users to items, with no direct connections among users themselves or items themselves. The adjacency matrix  $\mathbf{A}$  can be structured into four distinct blocks representing: User-User ( $\mathbf{A}_{UU}$ ), User-Item ( $\mathbf{A}_{UI}$ ), Item-User ( $\mathbf{A}_{IU}$ ), and Item-Item ( $\mathbf{A}_{II}$ ) interactions. This structure is illustrated in Figure 3, where  $|V_U|$  and  $|V_I|$  indicate the number of users and items.

We define a *simple path* as a path with no repeated nodes; for example,  $u_1 \rightarrow i_1 \rightarrow u_3 \rightarrow i_4$  is a simple path of length 3. **Question:** Identify all the simple paths of length exactly 3 between user  $u_1$  and item  $i_6$  based on Figure 3(a).

2. Define the propagation affinity  $S(u, i)$  as the sum of weights of all paths of length 3 between user  $u$  and item  $i$ . The weight of a single path  $W(p)$  is defined as:  $W(p) = \prod_{v \in p} \frac{1}{\deg(v)}$ , where  $v$  is the intermediate nodes (excluding the start and end nodes) in the path  $p$  and  $\deg(v)$  is the degree of node  $v$ .

**Question:** Calculate the exact value of  $S(u_1, i_6)$ . Besides, represent the propagation affinity matrix  $\mathbf{S} \in \mathbb{R}^{|V_U| \times |V_I|}$ , where each element represents the affinity score between a user-item pair, using  $\mathbf{D}$ ,  $\mathbf{A}$ , and  $\tilde{\mathbf{A}}$  (see Lecture 3, slides 52-53).

3. Recommendation can be framed as a link prediction task. Typically, a GNN is applied to learn embeddings for each node in the bipartite graph, and the affinity score  $\hat{y}_{u,i}$  is calculated via the dot-product of the user and item embeddings. A common starting point is treating the task as binary classification using the BCE loss. Here, existing edges are treated as positive labels ( $y_{u,i} = 1$ ), and non-existent edges are sampled as negative labels ( $y_{u,i} = 0$ ):

$$\mathcal{L}_{\text{BCE}} = \sum_{(u,i)} \ln \sigma(y_{u,i} \log(\hat{y}_{u,i}) + (1 - y_{u,i}) \log(1 - \hat{y}_{u,i}))$$

In modern recommendation systems, BPR loss is often preferred. It is a pair-wise loss function designed to optimize the relative ranking of items by maximizing the margin between a positive item  $i$  and a negative item  $j$  for a given user  $u$ :

$$\mathcal{L}_{\text{BPR}} = - \sum_{(u,i,j)} \ln \sigma(\hat{y}_{u,i} - \hat{y}_{u,j})$$

where  $\hat{y}_{u,i}$  is the predicted score for the interaction between user  $u$  and item  $i$ , and  $j$  is the sampled item that doesn't interact with  $u$ . We can find that BPR loss optimizes the ranking of items by encouraging the model to score interacted items higher than non-interacted ones. This focus on pairwise comparisons aligns better with the goal of recommendation systems, which is to rank relevant items higher rather than predicting absolute relevance scores.

**Question:** Given a certain positive and negative pair  $(u, i, j)$ , derive the gradient of the BPR loss with respect to the predicted score  $\hat{y}_{u,i}$ , i.e.,  $\frac{\partial \ln \sigma(\hat{y}_{u,i} - \hat{y}_{u,j})}{\partial \hat{y}_{u,i}}$ .